

Answer #01:

a. Synthesized (S)

E.val depends on E1.val and T.val (children)

b. Synthesized (S)

T.type depends on T1.type (child)

c. Inherited (L)

id.type gets value from parent T.type

d. Inherited (L)

B.env gets value from parent A.env

e. Synthesized (S)

E.val depends on E1.val (child)

f. Synthesized (S)

S.ok depends on id.type and E.type (children)

g. $\Rightarrow$ L.type $\rightarrow$ passed to id $\rightarrow$ Inherited (L)

$\Rightarrow$ id.type $\rightarrow$ from parent L $\rightarrow$ Inherited (L)

$\Rightarrow$ L.len $\rightarrow$ computed from L1.len $\rightarrow$ Synthesized (S)

h. Inherited (L)

S.table gets value from D.table (sibling/parent)

i. Synthesized (S)

F.val depends on E.val (child)

j. $\Rightarrow$ C.in $\rightarrow$ comes from outside $\rightarrow$ Inherited (L)

$\Rightarrow$ S1.in $\rightarrow$ from C.in $\rightarrow$ Inherited (L)

$\Rightarrow$ C.out $\rightarrow$ computed from E, S1.out, S2.out $\rightarrow$ Synthesized (S)

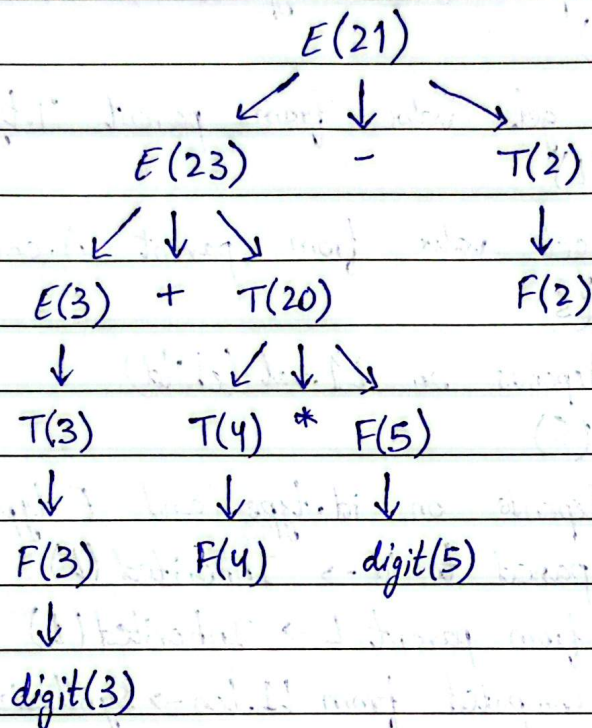
Answer #02:

a. Expression: $3 + 4 * 5 - 2$

Order: $\rightarrow 4 * 5 = 20$

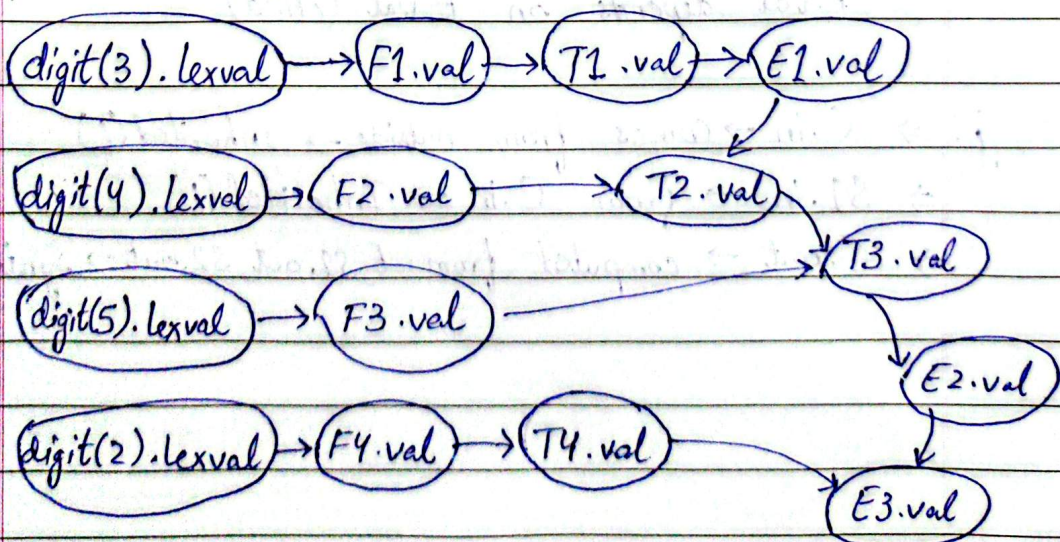
$\rightarrow 3 + 20 = 23$

$\rightarrow 23 - 2 = 21$



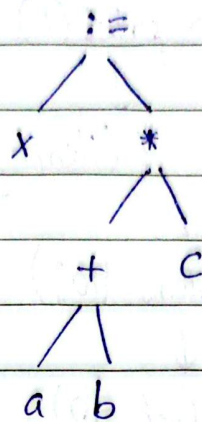
Final root value : 21

b. Dependency Graph:

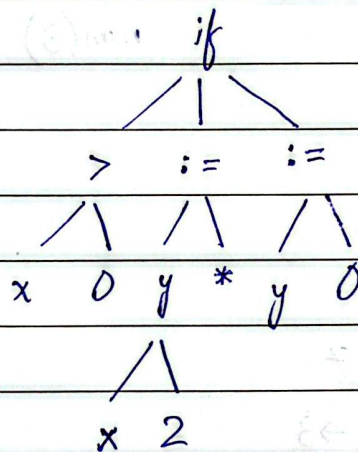


Answer #03:

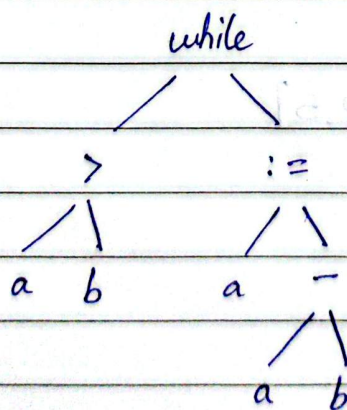
a. $x := (a+b) * c$



b. if $x > 0$ then $y := x * 2$ else $y := 0$

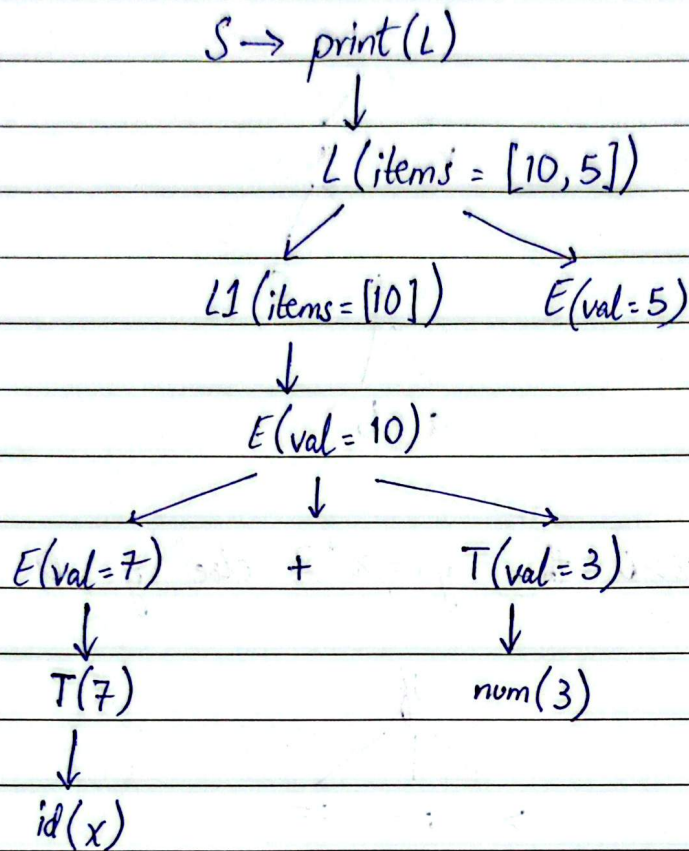


c. while $a > b$ do $a := a - b$



Answer #04:

a.



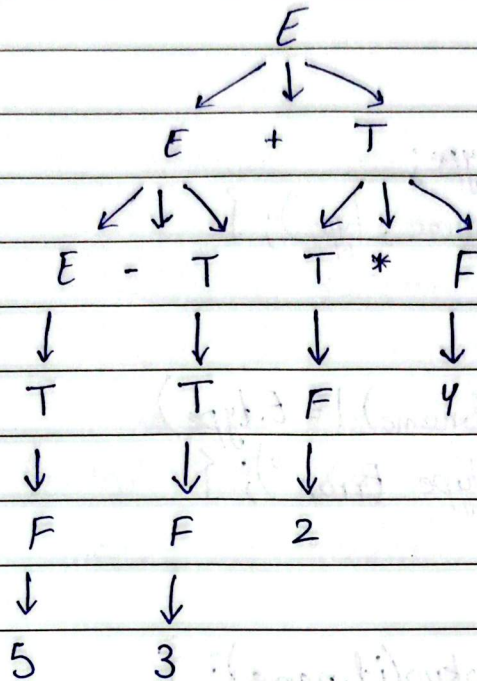
b.

 $x = 7$ $\Rightarrow id(x) \rightarrow 7$ $\Rightarrow num(3) \rightarrow 3$ $\Rightarrow E = 7 + 3 = 10$ $\Rightarrow L1.items = [10]$ $\Rightarrow num(5) \rightarrow 5$ $\Rightarrow E = 5$ $\Rightarrow L.items = [10, 5]$

Output:

10 5

Answer #05:

a. Expression: $5 - 3 + 2 * 4$ 

Order:

 $F \rightarrow \text{digit}(5) \quad \{\text{print}(5)\} \quad \text{Output: } 5$ $F \rightarrow \text{digit}(3) \quad \{\text{print}(3)\} \quad \text{Output: } 5 \ 3$ $E \rightarrow E1 - T \quad \{\text{print}(' - ')\} \quad \text{Output: } 5 \ 3 \ -$ $F \rightarrow \text{digit}(2) \quad \{\text{print}(2)\} \quad \text{Output: } 5 \ 3 \ - \ 2$ $F \rightarrow \text{digit}(4) \quad \{\text{print}(4)\} \quad \text{Output: } 5 \ 3 \ - \ 2 \ 4$ $T \rightarrow T1 * F \quad \{\text{print}(' * ')\} \quad \text{Output: } 5 \ 3 \ - \ 2 \ 4 \ *$ $E \rightarrow E1 + T \quad \{\text{print}(' + ')\} \quad \text{Output: } 5 \ 3 \ - \ 2 \ 4 \ * \ +$

Output:

 $5 \ 3 \ - \ 2 \ 4 \ * \ +$

6. SDT scheme:

$P \rightarrow D ; S$

$D \rightarrow \text{type id}$

{ id.type = type;
insert(id.name, type); }

$S \rightarrow \text{id} := E$

{ if (lookup(id.name) != E.type)
print("Type Error"); }

$E \rightarrow \text{id}$

{ E.type = lookup(id.name); }

$E \rightarrow \text{num}$

{ E.type = "int"; }

$E \rightarrow E1 + E2$

{ if (E1.type == "int" && E2.type == "int")
E.type = "int";

else

print("Type Error"); }

K232001

c.

(i)

SDD: defines attributes and rules

SDT: embeds actions directly in production

(ii)

When the semantic actions are executed after the complete production has been reduced.

- grammar should be suitable for bottom-up parsing.
- semantic actions should depend only on information already available on the parser stack.
- no inherited attributes requiring $L \rightarrow R$ information flow should be needed.

(iii)

No, not every SDD can be converted to SDT. Some SDDs require complex attribute dependencies that cannot be enforced by fixed action positions.